

Smart Lender

Project Description:

The **Smart Lender, a Loan Approval Prediction System** is a Machine Learning-based web application designed to automate the loan approval process by predicting whether a loan application is likely to be approved or rejected based on the applicant's financial and personal details. The system helps banks and financial institutions make faster, more accurate, and unbiased lending decisions by analyzing historical loan data and identifying patterns associated with successful loan approvals.

The application utilizes multiple Machine Learning algorithms, including **Decision Tree**, **Random Forest**, **K-Nearest Neighbors (KNN)**, and **Gradient Boosting (XGBoost)**, to train predictive models. After evaluating the performance of each model using classification metrics and cross-validation, the best-performing model is selected and integrated into a Flask-based web application. Users can enter applicant information such as gender, marital status, education, income, loan amount, loan term, credit history, and property area through a simple web interface to receive an instant loan approval prediction.

The project is developed using **Python**, **Flask**, **Scikit-learn**, **Pandas**, **NumPy**, **Matplotlib**, and **Seaborn**. It follows a structured Machine Learning workflow consisting of data collection, exploratory data analysis, preprocessing, model training, evaluation, and deployment. By reducing manual effort and improving decision consistency, the Smart Lender system serves as an intelligent decision-support tool for the banking and financial sector.

Scenarios

Scenario 1: Loan Eligibility Check for a Salaried Employee

A salaried applicant visits the Smart Lender application and enters details such as monthly income, education, marital status, loan amount, loan term, and credit history. The trained Machine Learning model analyzes the information and instantly predicts whether the loan application is likely to be approved, enabling the applicant to understand their eligibility before submitting a formal request.

Scenario 2: Bank Loan Officer Decision Support

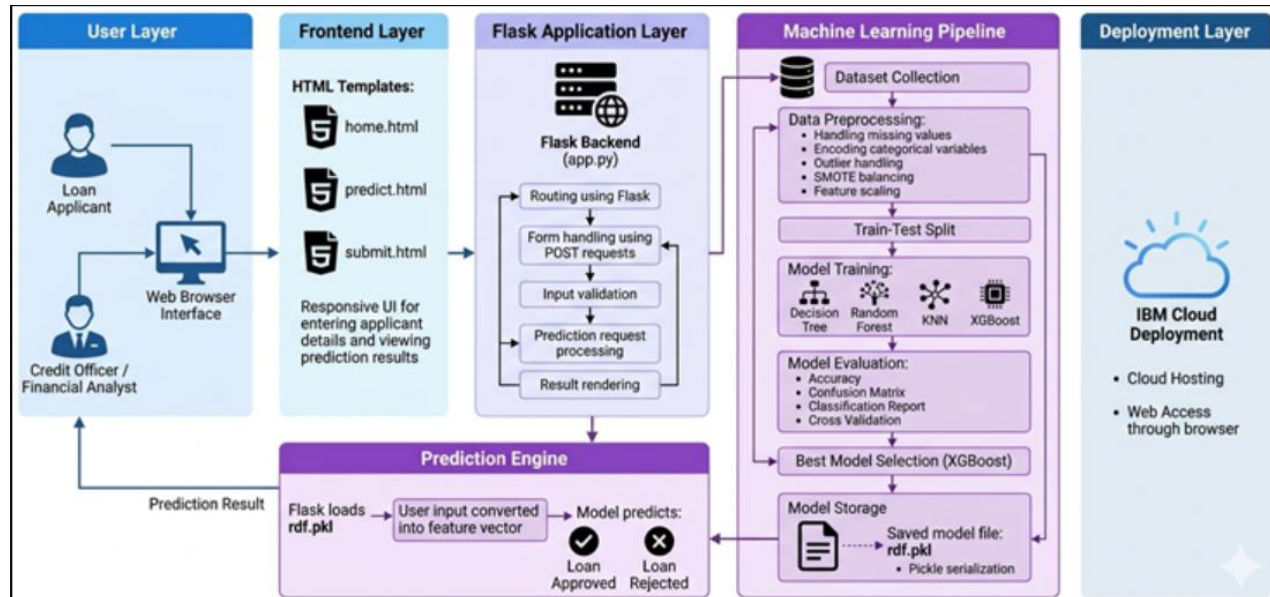
A loan officer receives multiple loan applications every day. Instead of manually reviewing each application, the officer enters the applicant's information into the Smart Lender system. The application provides a prediction based on historical loan data, helping the officer make quicker, data-driven decisions while reducing processing time and human bias.

Scenario 3: Financial Institution Risk Assessment

A financial institution uses the Smart Lender system as a preliminary screening tool for new loan applications. The application evaluates applicant details using the trained Machine

Learning model and predicts the likelihood of loan approval. This allows the institution to identify high-risk applications early and prioritize detailed verification only for selected cases, improving operational efficiency and reducing credit risk.

Technical Architecture:



Description: The Smart Lender – Loan Approval Prediction System utilizes a modular three-tier architecture to provide fast and accurate loan eligibility predictions. The frontend is developed using HTML and CSS to collect applicant information through a simple and responsive user interface, while the Flask-based backend manages user requests, data validation, model loading, and prediction processing. The backend communicates with a trained Machine Learning model developed using Scikit-learn, which analyzes applicant details and predicts loan approval based on historical data. The application is deployed locally using the Flask development server, ensuring an efficient, reliable, and user-friendly loan prediction experience.

Pre-requisites

- **Python Programming Knowledge: Python Documentation**
- **Flask Framework: Flask Documentation**
- **Machine Learning Basics: Scikit-learn Documentation**
- **Pandas and NumPy: Python Libraries Documentation**
- **HTML, CSS, and Bootstrap: W3Schools Tutorials**
- **Development Environment: Anaconda Navigator / PyCharm**
- **Version Control with Git: Git Documentation**

Project Workflow:

Milestone 1: Data Collection and Architecture Design

- **Activity 1.1:** Download the loan prediction dataset from the provided source.
- **Activity 1.2:** Study the dataset and identify the input and target variables.
- **Activity 1.3:** Design the system architecture and machine learning workflow.
- **Activity 1.4:** Set up the development environment by installing the required libraries and dependencies.

Milestone 2: Data Analysis and Preprocessing

- **Activity 2.1:** Import the dataset and perform exploratory data analysis.
- **Activity 2.2:** Perform univariate, bivariate, and multivariate analysis.
- **Activity 2.3:** Handle missing values, encode categorical features, and balance the dataset using SMOTE.
- **Activity 2.4:** Scale the features and split the dataset into training and testing sets.

Milestone 3: Model Building

- **Activity 3.1:** Train multiple machine learning models such as Decision Tree, Random Forest, K-Nearest Neighbors (KNN), and Gradient Boosting.
- **Activity 3.2:** Evaluate the models using performance metrics and select the best-performing model

Milestone 4: Flask Application Development

- **Activity 4.1:** Develop the Flask backend and integrate the trained machine learning model.
- **Activity 4.2:** Design the user interface using HTML and CSS for loan prediction.

Milestone 5: Deployment

- **Activity 5.1:** Configure the application for local deployment.
- **Activity 5.2:** Test the application and verify prediction results.

Milestone 6: Conclusion

Milestone 1: Data Collection and Architecture Design

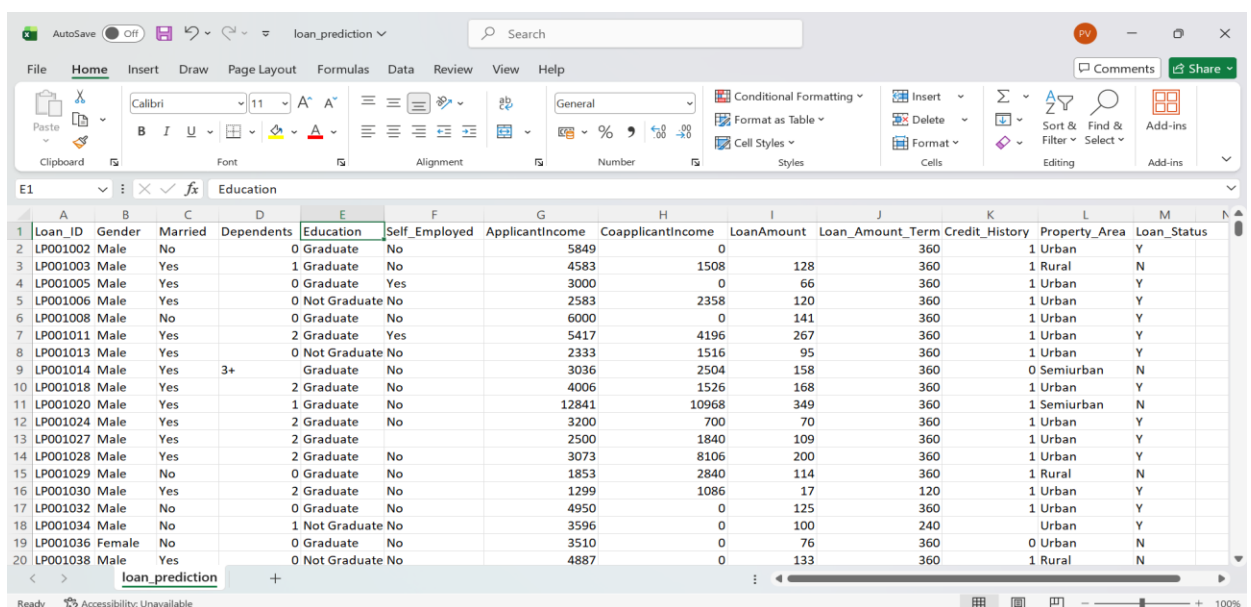
In this milestone, the required loan prediction dataset is collected and analyzed to understand its features and target variable. The application architecture is designed to define the interaction between the user interface, Flask backend, and the machine learning model. The development environment is then configured by installing the required Python libraries and tools for data analysis, model training, and application development.

Activity 1.1: Download the Loan Prediction Dataset

Before building the machine learning model, download the loan prediction dataset from the provided Google Drive link and store it in the project directory.

Step 1 – Open the Dataset Link: Visit the Google Drive link provided for the Loan Prediction dataset.

Step 2 – Download the Dataset: Download the dataset file (CSV format) to your local system.



Loan_ID	Gender	Married	Dependents	Education	Self_Employed	ApplicantIncome	CoapplicantIncome	LoanAmount	Loan_Amount_Term	Credit_History	Property_Area	Loan_Status
LP001002	Male	No	0	Graduate	No	5849	0	360	360	1	Urban	Y
LP001003	Male	Yes	1	Graduate	No	4583	1508	128	360	1	Rural	N
LP001005	Male	Yes	0	Graduate	Yes	3000	0	66	360	1	Urban	Y
LP001006	Male	Yes	0	Not Graduate	No	2583	2358	120	360	1	Urban	Y
LP001008	Male	No	0	Graduate	No	6000	0	141	360	1	Urban	Y
LP001011	Male	Yes	2	Graduate	Yes	5417	4196	267	360	1	Urban	Y
LP001013	Male	Yes	0	Not Graduate	No	2333	1516	95	360	1	Urban	Y
LP001014	Male	Yes	3+	Graduate	No	3036	2504	158	360	0	Semiurban	N
LP001018	Male	Yes	2	Graduate	No	4006	1526	168	360	1	Urban	Y
LP001020	Male	Yes	1	Graduate	No	12841	10968	349	360	1	Semiurban	N
LP001024	Male	Yes	2	Graduate	No	3200	700	70	360	1	Urban	Y
LP001027	Male	Yes	2	Graduate	No	2500	1840	109	360	1	Urban	Y
LP001028	Male	Yes	2	Graduate	No	3073	8106	200	360	1	Urban	Y
LP001029	Male	No	0	Graduate	No	1853	2840	114	360	1	Rural	N
LP001030	Male	Yes	2	Graduate	No	1299	1086	17	120	1	Urban	Y
LP001032	Male	No	0	Graduate	No	4950	0	125	360	1	Urban	Y
LP001034	Male	No	1	Not Graduate	No	3596	0	100	240	0	Urban	Y
LP001036	Female	No	0	Graduate	No	3510	0	76	360	0	Urban	N
LP001038	Male	Yes	0	Not Graduate	No	4887	0	133	360	1	Rural	N

Step 3 Create the Project Folder: Create a folder named Smart Lender and organize the project files within it.

Step 4 Place the Dataset: Copy the downloaded dataset into the project folder so it can be accessed during the data preprocessing and model training .

Step 5 Verify the Dataset: Ensure that the dataset file is available in the project directory and can be successfully loaded using the Pandas library before proceeding with the next activities.

Step 6 Prepare the Dataset for Analysis: Ensure that the dataset is stored in the correct project directory and is ready to be imported into the Python environment for data exploration, preprocessing, and model development.

Important — Keep the original dataset unchanged and use a separate copy if any modifications are required during preprocessing. This helps maintain data integrity and allows the original data to be reused whenever necessary.

Activity 1.2 Study and Understand the Loan Prediction Dataset

Before building the machine learning model, it is important to understand the dataset and identify the features that influence loan approval. The following steps are performed to analyze the dataset before preprocessing and model development.

- **Understand the Dataset:** Review the dataset to identify the input features and the target variable used for loan prediction.
- **Identify Feature Types:** Classify the features as numerical or categorical to determine the appropriate preprocessing techniques.
- **Analyze the Target Variable:** Examine the distribution of the loan approval status to understand the class balance in the dataset.
- **Study Feature Relationships:** Analyze how applicant details such as income, credit history, education, loan amount, and property area influence loan approval.
- **Prepare for Preprocessing:** Identify missing values, duplicate records, and inconsistent data that must be handled before training the machine learning models.

Activity 1.3 Define the Architecture of the Application

- **Draft the System Architecture:** Create a system architecture diagram illustrating the interaction between the user interface, Flask backend, and the trained machine learning model.
- **Define the User Interface:** Design a simple and responsive interface where users can enter applicant details such as gender, marital status, education, income, loan amount, loan amount term, credit history, and property area.
- **Outline Backend Responsibilities:** Specify how the Flask application receives user inputs, validates the data, loads the trained machine learning model, processes the prediction request, and returns the result to the user.
- **Describe Model Integration:** Define how the trained machine learning model is loaded using the Pickle library, processes the applicant details, predicts the loan approval status, and sends the prediction back to the Flask application for display.

Activity 1.4: Set Up the Development Environment

Before starting the development of the Smart Lender Loan Approval Prediction System, the required software tools and Python libraries were installed and configured. The project was developed using **Visual Studio Code (VS Code)** as the development environment and **Flask** as the web application framework.

Step 1: Install Python

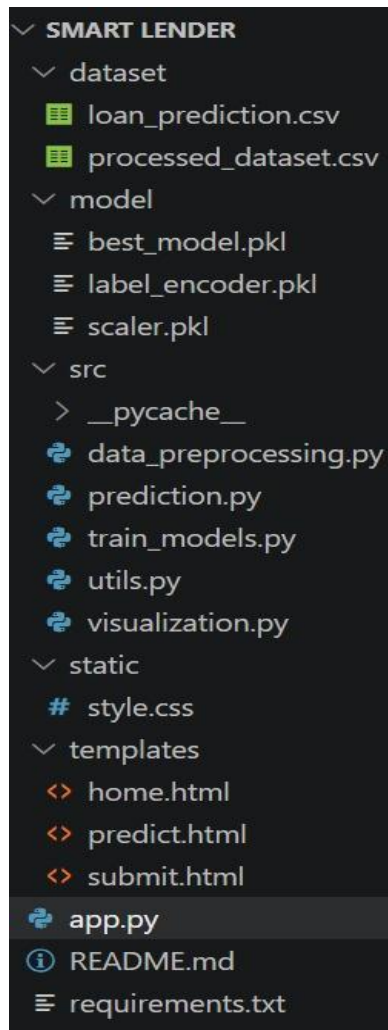
Python 3.10 or above was installed on the system since the entire project was implemented using the Python programming language.

Step 2: Install Visual Studio Code

Visual Studio Code (VS Code) was installed and used as the Integrated Development Environment (IDE). VS Code provides features such as syntax highlighting, debugging, IntelliSense, and integrated terminal support, making Python and Flask development easier.

Step 3: Create the Project Folder

A project folder named **SMART_LENDER** was created to organize all project files and folders.



Step 4: Install the Required Libraries

The required Python libraries were installed using the VS Code integrated terminal.

```
pip install numpy pandas matplotlib seaborn scikit-learn imbalanced-learn flask joblib
```

```
pip install xgboost
```

Step 5: Install All Dependencies

After creating the **requirements.txt** file, all dependencies were installed using the following command.

Step 6: Open the Project in VS Code

The Smart Lender project folder was opened in Visual Studio Code. The integrated terminal was used to execute Python scripts, train machine learning models, and run the Flask application.

Step 7: Verify the Dataset

The loan prediction dataset was placed inside the **dataset** folder and loaded using the Pandas library.

Code:

```
import pandas as pd

df = pd.read_csv("dataset/loan_prediction.csv")

print(df.head())
```

Step 8: Train and Save the Machine Learning Model

After preprocessing the dataset and training multiple machine learning algorithms (Decision Tree, Random Forest, KNN, and Gradient Boosting), the best-performing model along with the scaler and label encoder were saved in the **model** folder.

Saved Files in Folder:

best_model.pkl

scaler.pkl

label_encoder.pkl

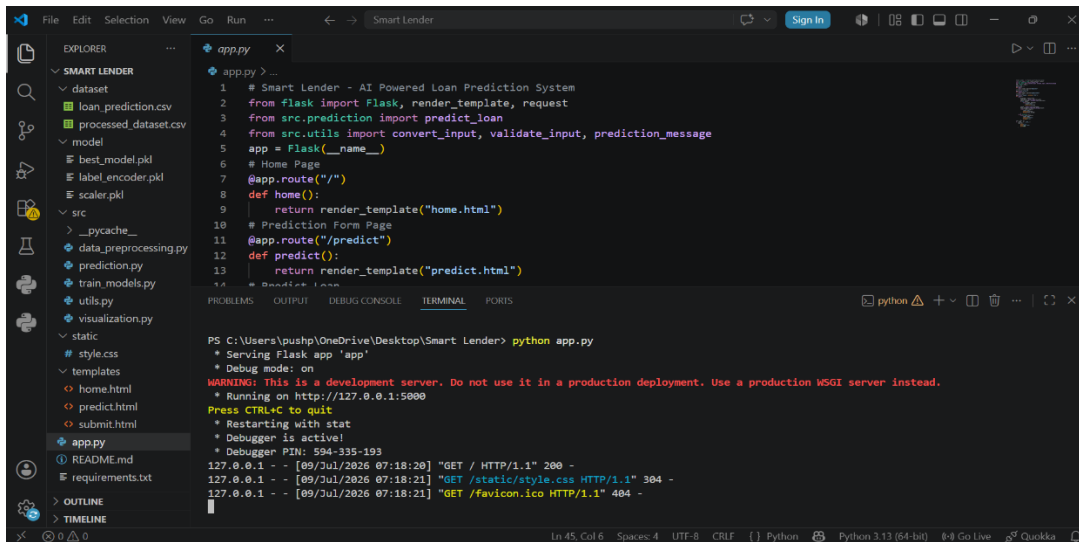
Step 9: Run the Flask Application

The Flask application was executed from the VS Code terminal using the following command.

```
python app.py
```

After the Flask server started successfully, the application was accessed through the local web browser using:

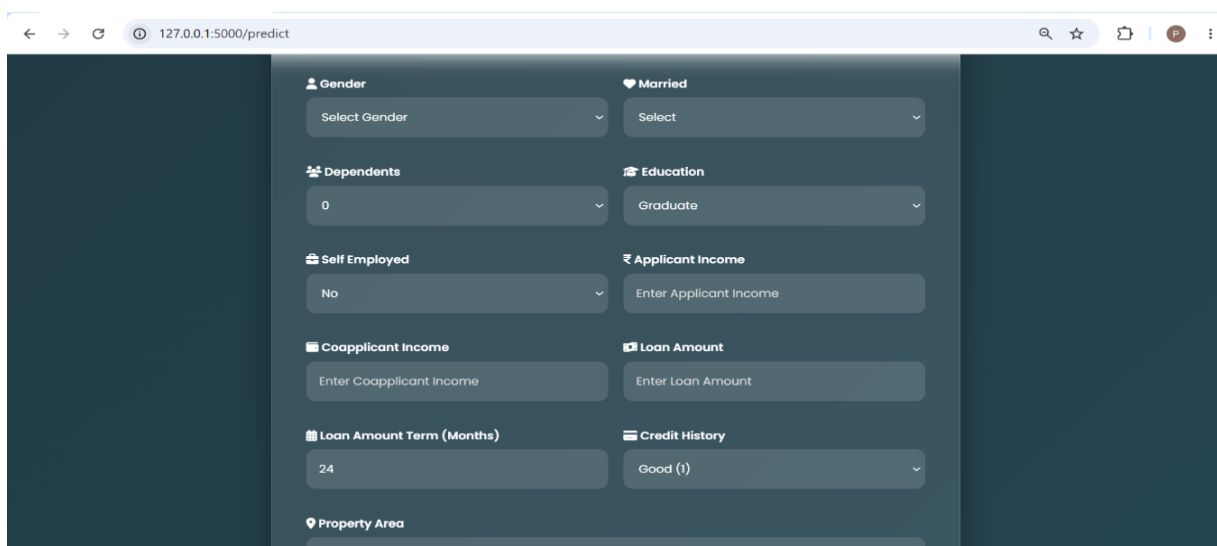
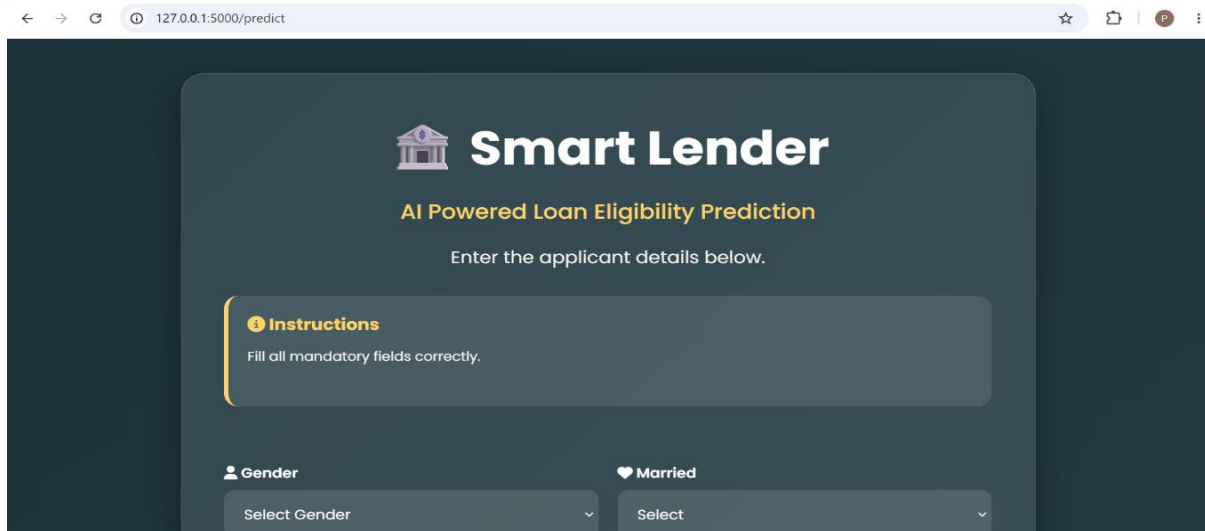
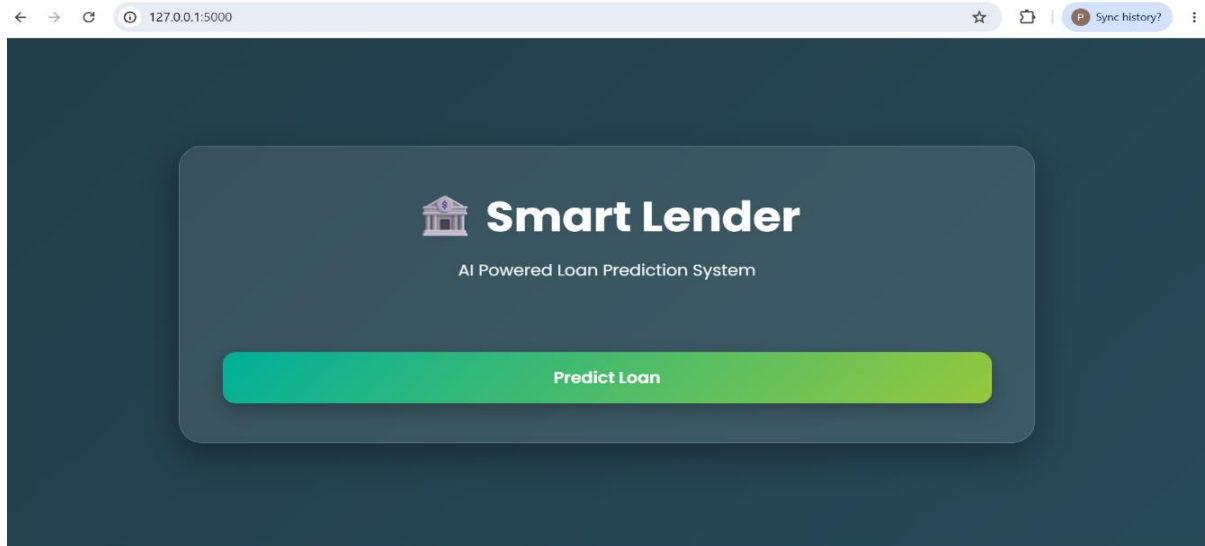
<http://127.0.0.1:5000>

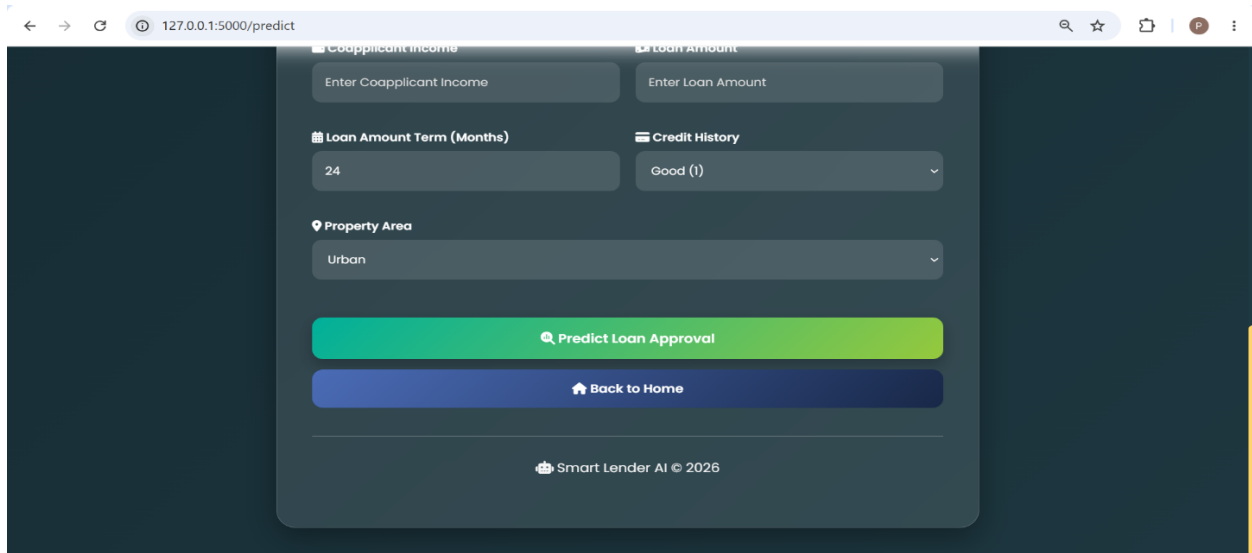


The screenshot shows the Visual Studio Code editor with a project named 'Smart Lender'. The Explorer sidebar on the left shows the file structure, including 'dataset', 'model', 'src', and 'static' folders. The main editor window displays the code for 'app.py', which is a Flask application for a loan prediction system. The code includes imports for Flask, render_template, request, and prediction utilities, and defines routes for the home page and a prediction endpoint. The bottom panel shows the terminal output, which confirms that the application is running on http://127.0.0.1:5000 and displays some initial GET requests.

```
1 # Smart Lender - AI Powered Loan Prediction System
2 from flask import Flask, render_template, request
3 from src.prediction import predict_loan
4 from src.utils import convert_input, validate_input, prediction_message
5 app = Flask(__name__)
6 # Home Page
7 @app.route("/")
8 def home():
9     return render_template("home.html")
10 # Prediction Form Page
11 @app.route("/predict")
12 def predict():
13     return render_template("predict.html")
14 # predict loan
```

```
PS C:\Users\pushp\OneDrive\Desktop\Smart Lender> python app.py
* Serving Flask app 'app'
* Debug mode: on
WARNING: This is a development server. Do not use it in a production deployment. Use a production WSGI server instead.
* Running on http://127.0.0.1:5000
Press CTRL+C to quit
* Restarting with stat
* Debugger is active!
* Debugger PIN: 594-335-193
127.0.0.1 - - [09/Jul/2026 07:18:20] "GET / HTTP/1.1" 200 -
127.0.0.1 - - [09/Jul/2026 07:18:21] "GET /static/style.css HTTP/1.1" 304 -
127.0.0.1 - - [09/Jul/2026 07:18:21] "GET /favicon.ico HTTP/1.1" 404 -
```





127.0.0.1:5000/predict

Coapplicant Income: Enter Coapplicant Income

Loan Amount: Enter Loan Amount

Loan Amount Term (Months): 24

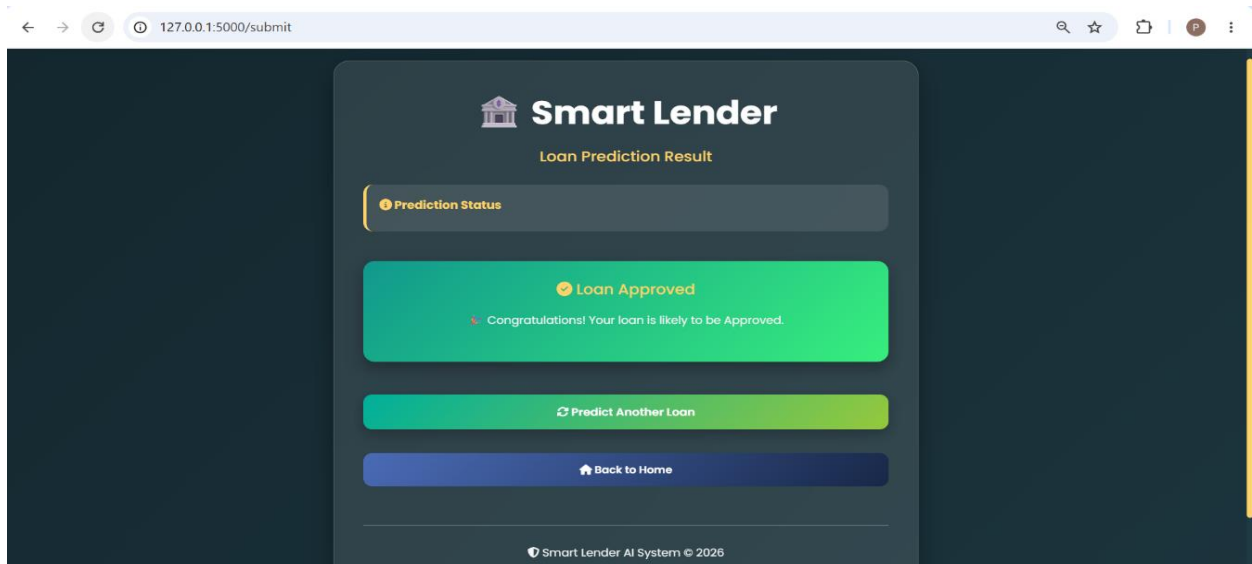
Credit History: Good (1)

Property Area: Urban

[Predict Loan Approval](#)

[Back to Home](#)

Smart Lender AI © 2026



127.0.0.1:5000/submit

Smart Lender

Loan Prediction Result

Prediction Status

Loan Approved

Congratulations! Your loan is likely to be Approved.

[Predict Another Loan](#)

[Back to Home](#)

Smart Lender AI System © 2026

Users entered the applicant details on the prediction page, and the trained machine learning model generated the loan approval prediction.

Step 10: Development Environment Ready

After successfully installing all software tools, Python libraries, configuring the project structure, and running the Flask application, the Smart Lender system was ready for data analysis, machine learning model training, and loan prediction.

Milestone 2: Data Analysis and Preprocessing

Activity 2.1: Import and Analyze the Dataset

Import the loan prediction dataset and perform exploratory data analysis to understand the characteristics of the data before building the machine learning model.

- Import the required Python libraries such as Pandas, NumPy, Matplotlib, and Seaborn.
- Read the loan prediction dataset using the **read_csv()** function.
- Display the first few records of the dataset using **head()** to verify that the dataset has been loaded correctly.
- Examine the dataset using **shape()**, **info()**, and **describe()** to understand its structure and statistical information.

Activity 2.2: Perform Exploratory Data Analysis

Perform exploratory data analysis to understand the distribution of features and identify relationships between variables.

Visualize the Dataset:

- Perform **Univariate Analysis** to examine the distribution of individual features using count plots and distribution plots.
- Perform **Bivariate Analysis** to identify relationships between two variables using count plots.
- Perform **Multivariate Analysis** using swarm plots to understand the relationship among multiple features.
- Record the observations obtained from the visualizations to assist in data preprocessing and model building.

Milestone 3: Model Building

In this milestone, multiple machine learning classification algorithms are trained and evaluated to identify the most suitable model for predicting loan approval. The preprocessed dataset is used to train the models after handling missing values, encoding categorical variables, balancing the dataset using SMOTE, scaling the features, and splitting the dataset into training and testing sets.

Each algorithm is trained using the training dataset and evaluated on the testing dataset using performance metrics such as Accuracy Score, Confusion Matrix, Classification Report, and Cross-Validation. The performance of all models is compared, and the best-performing model is selected and saved for integration into the Flask web application.

The following machine learning algorithms are implemented during this milestone:

- Decision Tree Classifier
- Random Forest Classifier
- K-Nearest Neighbors (KNN) Classifier
- Gradient Boosting Classifier (or XGBoost, if implemented)

The selected model is serialized into a **Pickle (.pkl)** file, which is later loaded by the Flask application to provide real-time loan approval predictions.

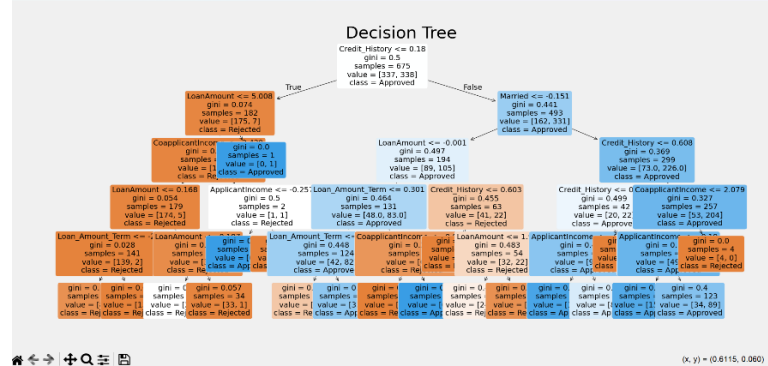
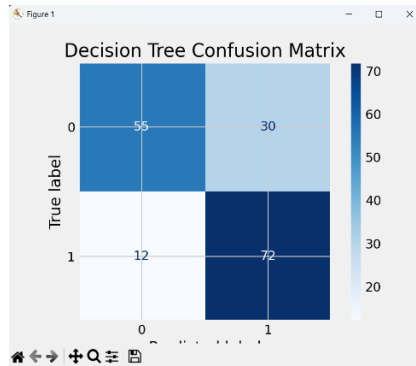
Activity 3.1: Train the Decision Tree Model

The Decision Tree algorithm is one of the most widely used supervised learning algorithms for classification problems. It constructs a tree-like model by recursively splitting the dataset based on the feature that provides the highest information gain.

In this project, the Decision Tree model is trained using the processed training dataset. After training, the model predicts the loan approval status of the testing dataset. The model performance is evaluated using the confusion matrix, classification report, and accuracy score.

Steps Performed

- Import the DecisionTreeClassifier.
- Create the Decision Tree model.
- Train the model using the training dataset.
- Predict loan approval on the testing dataset.
- Evaluate model performance using Accuracy Score, Confusion Matrix, and Classification Report.



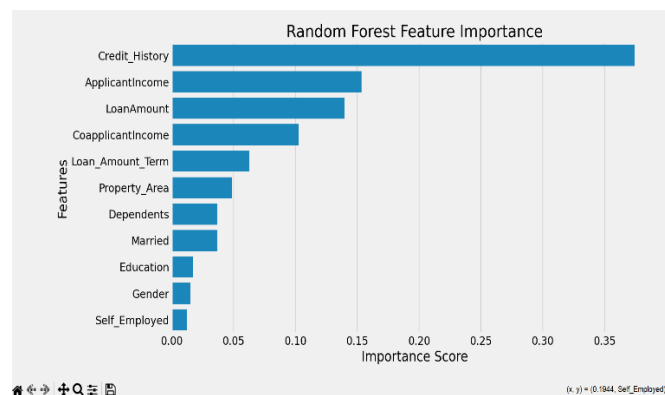
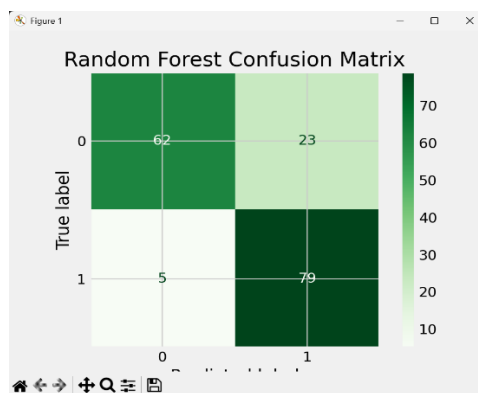
Activity 3.2: Train the Random Forest Model

Random Forest is an ensemble machine learning algorithm that combines multiple Decision Trees to improve prediction accuracy and reduce overfitting.

The model is trained using the same training dataset and tested using the testing dataset. Predictions generated by the Random Forest model are compared with the actual loan approval values.

Steps Performed

- Import the RandomForestClassifier.
- Initialize the Random Forest model.
- Train the model.
- Generate predictions.
- Evaluate using Accuracy Score, Confusion Matrix, and Classification Report.



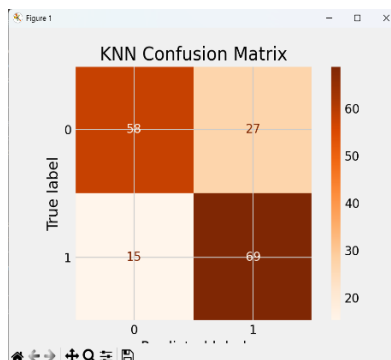
Activity 3.3: Train the K-Nearest Neighbors (KNN) Model

K-Nearest Neighbors (KNN) is a distance-based classification algorithm that classifies new observations based on the majority class of their nearest neighboring data points.

Since KNN depends on distance calculations, feature scaling is performed before training the model.

Steps Performed

- Import the KNeighborsClassifier.
- Create the KNN model.
- Train the model using the scaled training data.
- Predict loan approval.
- Evaluate the prediction results.



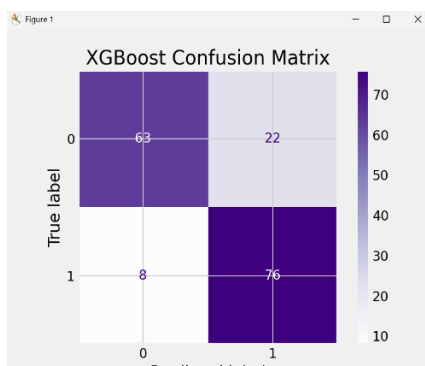
Activity 3.4: Train the Gradient Boosting Model

Gradient Boosting is an ensemble learning algorithm that builds multiple weak learners sequentially. Each new model attempts to reduce the prediction errors made by the previous model, resulting in improved prediction accuracy.

The model is trained using the training dataset and evaluated on the testing dataset.

Steps Performed

- Import the GradientBoostingClassifier.
- Initialize the Gradient Boosting model.
- Train the model.
- Predict the testing dataset.
- Evaluate model performance.



Activity 3.5: Model Evaluation and Selection

After training all machine learning models, their performances are compared to determine the most accurate classifier for loan approval prediction.

The evaluation is performed using the following performance metrics:

- Accuracy Score
- Confusion Matrix
- Classification Report
- Precision
- Recall
- F1-Score
- Cross-Validation (5-Fold)

The model achieving the highest prediction accuracy and the best overall performance is selected as the final model for deployment.

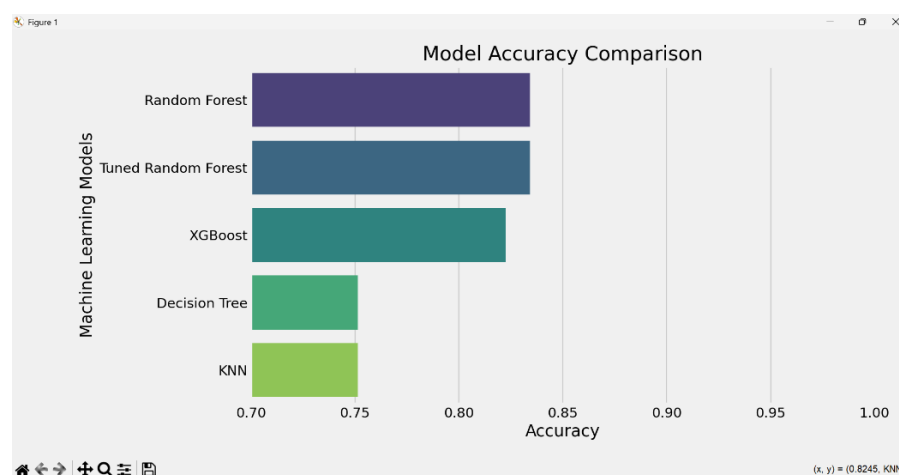
The selected model is saved using the Pickle library for later use in the Flask web application.

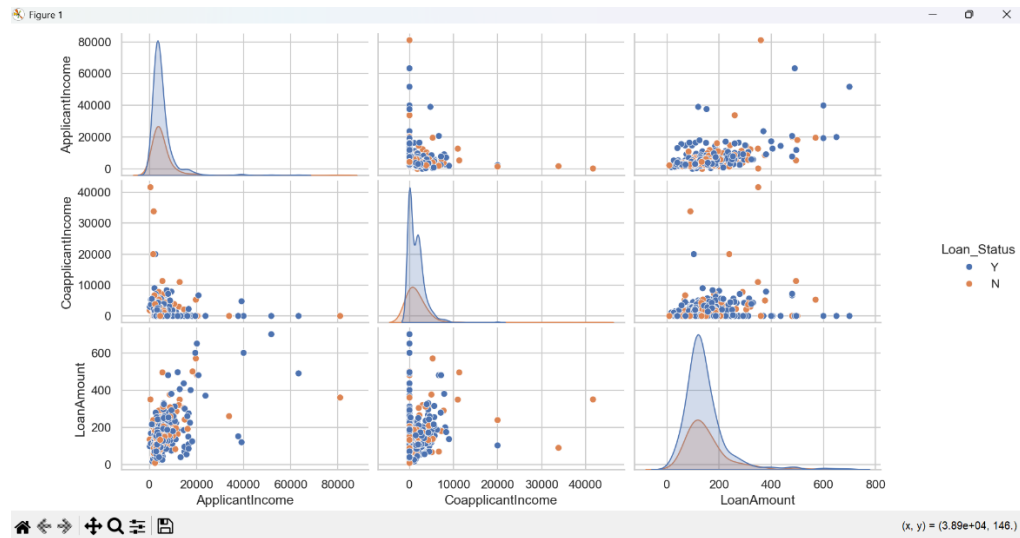
Code:

```
import pickle
pickle.dump(best_model, open("model/best_model.pkl", "wb"))
```

Saved files:

```
model/
|
├── best_model.pkl
├── scaler.pkl
└── label_encoder.pkl
```





Milestone 4: Frontend Development

Description

In this milestone, the trained machine learning model is integrated with the Flask web framework to create a user-friendly web application. Flask acts as the backend server, receiving applicant details entered through the web interface, processing the input using the trained machine learning model, and displaying the loan approval prediction. The frontend is developed using HTML and CSS to provide a simple, responsive, and interactive interface for users.

Activity 4.1: Develop the Flask Backend

The Flask backend is responsible for connecting the user interface with the trained machine learning model. It receives applicant information submitted through the HTML form, validates the input values, preprocesses the data if required, loads the trained model, and generates the loan approval prediction.

Steps Performed

- Import Flask and the required Python libraries.
- Load the trained machine learning model using the Pickle library.
- Create the Flask application instance.
- Configure application routes.

- Receive user inputs from the prediction form.
- Convert the input data into the required format.
- Pass the processed data to the trained machine learning model.
- Generate the prediction result.
- Display the prediction on the result page.

Output

- Flask application successfully connected with the trained machine learning model.
- Prediction results are generated dynamically based on user input.

Activity 4.2: Design the User Interface

A responsive web interface is developed using HTML and CSS to make the application easy to use. The interface allows users to enter applicant details required for loan prediction.

The application consists of three web pages:

Home Page (home.html)

The Home page serves as the landing page of the Smart Lender application. It introduces the project and provides a button to navigate to the loan prediction page.

Features:

- Project title
- Brief project description
- Predict Loan button
- Responsive design

Prediction Page (predict.html)

The Prediction page contains a form that collects applicant information required for loan approval prediction.

Input fields include:

- Gender
- Married
- Dependents
- Education
- Self Employed
- Applicant Income
- Co-Applicant Income
- Loan Amount
- Loan Amount Term
- Credit History
- Property Area

After entering all the details, the user clicks the **Submit** button to send the data to the Flask backend.

Result Page (submit.html)

The Result page displays the prediction generated by the machine learning model.

The prediction is displayed as either:

- **Loan Approved**
- **Loan Not Approved**

The page also provides an option to return to the prediction page for another prediction.

Activity 4.3: Validate the Application

The application is tested by entering different applicant details through the user interface.

Validation includes

- Verifying all input fields.
- Checking data submission.
- Confirming successful communication between Flask and the trained model.
- Displaying the correct prediction.
- Handling invalid or incomplete inputs gracefully.

Milestone 5: Deployment

Description

After successful integration of the machine learning model with the Flask application, the Smart Lender system is deployed locally and tested to ensure that the application functions correctly. During this milestone, all project components including the trained model, HTML templates, CSS files, dataset, and Flask application are verified. The application is then executed using the Flask development server and tested with different applicant data to validate prediction accuracy.

Activity 5.1: Prepare the Application for Deployment

Before running the application, all required project files are organized properly.

Project Structure

SMART_LENDER_PROJECT

```
dataset/  
model/  
src/  
templates/  
static/  
app.py  
requirements.txt  
README.md
```

The required Python libraries are installed using:
pip install -r requirements.txt

Activity 5.2: Run the Flask Application

The Flask application is executed from the VS Code terminal.

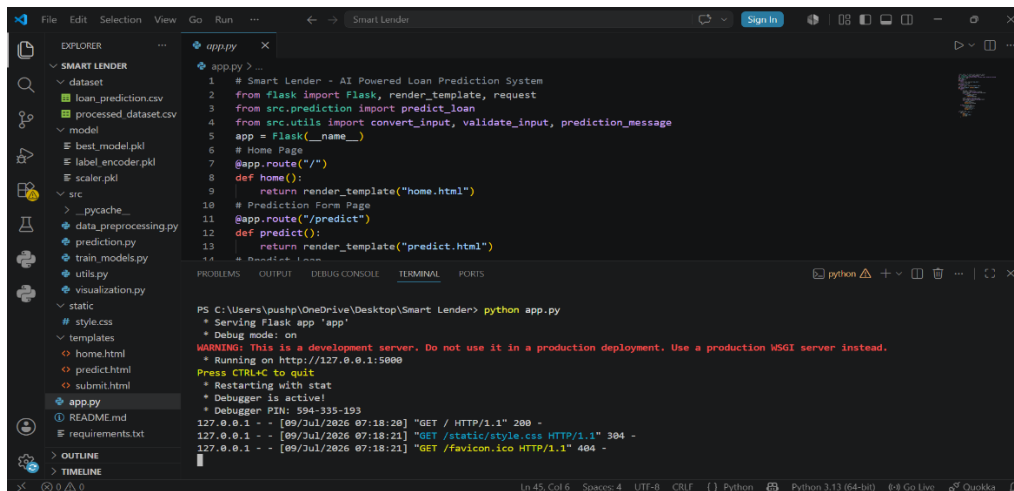
Command

`python app.py`

After successful execution, Flask starts the local development server.

Running on <http://127.0.0.1:5000/>

The application is opened in a web browser using the displayed URL.



The screenshot shows the VS Code interface with the 'app.py' file open. The code defines a Flask application with routes for home and prediction. The terminal output shows the command 'python app.py' being executed, resulting in a warning about development server usage and the application running on http://127.0.0.1:5000. It also shows several GET requests being handled successfully.

```

1 # Smart Lender - AI Powered Loan Prediction System
2 from flask import Flask, render_template, request
3 from src.prediction import predict_loan
4 from src.utils import convert_input, validate_input, prediction_message
5 app = Flask(__name__)
6 # Home Page
7 @app.route("/")
8 def home():
9     return render_template("home.html")
10 # Prediction Form Page
11 @app.route("/predict")
12 def predict():
13     return render_template("predict.html")
14 # Prediction Form Page

```

```

PS C:\Users\pushp\OneDrive\Desktop\Smart Lender> python app.py
* Serving Flask app 'app'
* Debug mode: on
WARNING: This is a development server. Do not use it in a production deployment. Use a production WSGI server instead.
* Running on http://127.0.0.1:5000
Press CTRL+C to quit
* Restarting with stat
* Debugger is active!
* Debugger PIN: 594-335-193
127.0.0.1 - - [09/Jul/2026 07:18:20] "GET / HTTP/1.1" 200 -
127.0.0.1 - - [09/Jul/2026 07:18:21] "GET /static/style.css HTTP/1.1" 304 -
127.0.0.1 - - [09/Jul/2026 07:18:21] "GET /favicon.ico HTTP/1.1" 404 -

```

Activity 5.3: Test the Application

Different applicant records are entered into the prediction form to verify that the application generates correct predictions.

Testing Steps

- Open the Smart Lender application.
- Navigate to the Prediction page.
- Enter applicant details.
- Click the Submit button.
- Observe the prediction result.
- Repeat the process with multiple records.

Activity 5.4: Verify Prediction Results

The prediction generated by the machine learning model is compared with the expected loan approval status to verify the correctness of the application.

The following aspects are verified:

- Correct loading of the trained model.
- Accurate preprocessing of user inputs.
- Successful prediction generation.
- Proper display of results.
- Smooth navigation between web pages.

Activity 5.5: Final Deployment Outcome

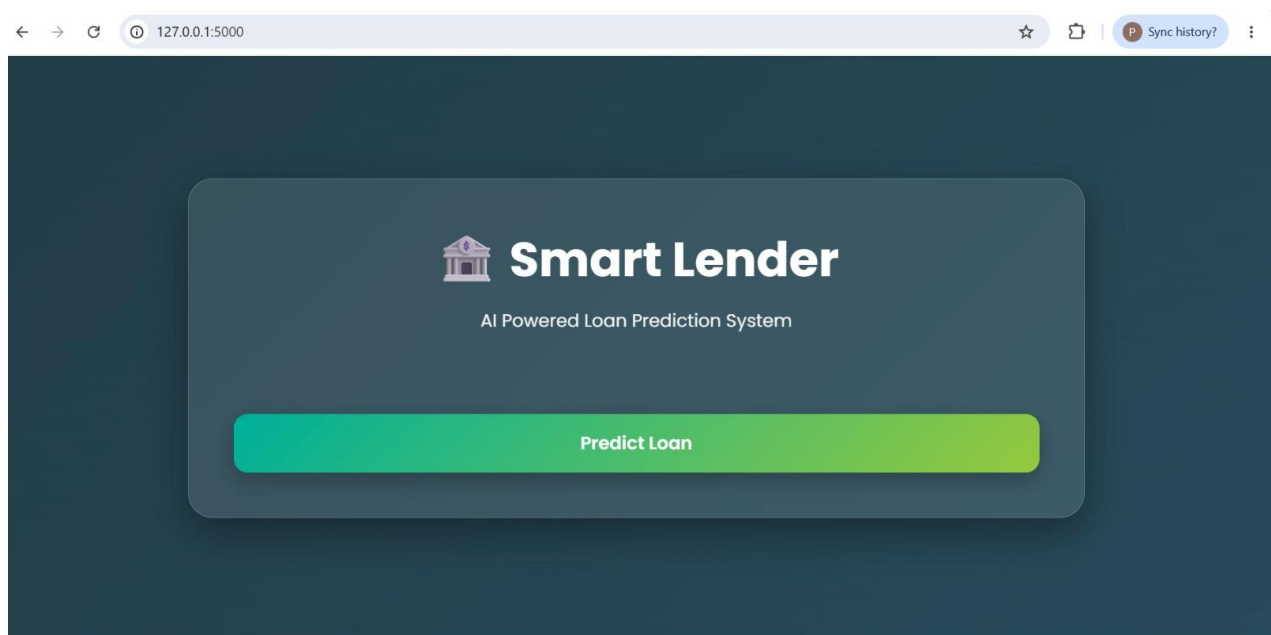
After successful testing, the Smart Lender application is able to:

- Accept applicant details through the web interface.
- Process the data using the trained machine learning model.
- Predict loan approval status.
- Display the prediction instantly.
- Provide a simple and user-friendly interface for end users.

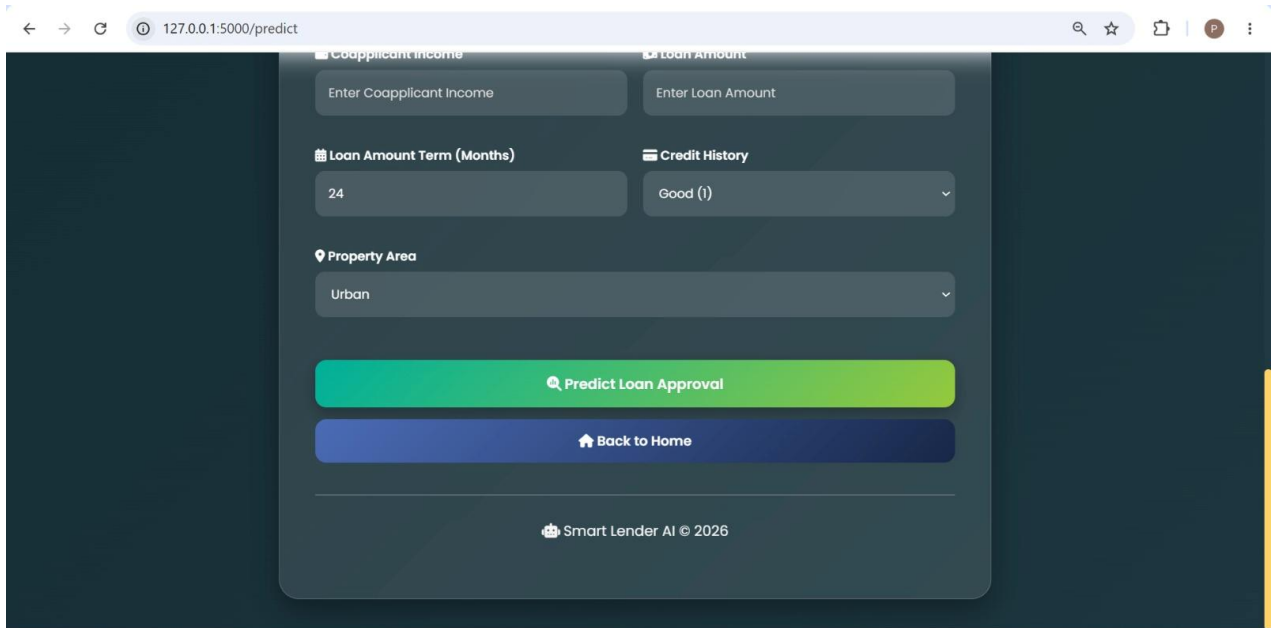
The application successfully demonstrates the practical implementation of Artificial Intelligence and Machine Learning techniques for loan approval prediction using Flask.

Exploring the Website's Web Pages:

Home / Generator Page:



Description: The Home Page serves as the landing page of the Smart Lender application. It provides a simple and user-friendly interface with the project title and a brief description of the loan approval prediction system. The page includes a **Predict** button that navigates users to the loan prediction form, allowing them to enter applicant details and access the prediction service.



127.0.0.1:5000/predict

Coapplicant Income: Enter Coapplicant Income

Loan Amount: Enter Loan Amount

Loan Amount Term (Months): 24

Credit History: Good (I)

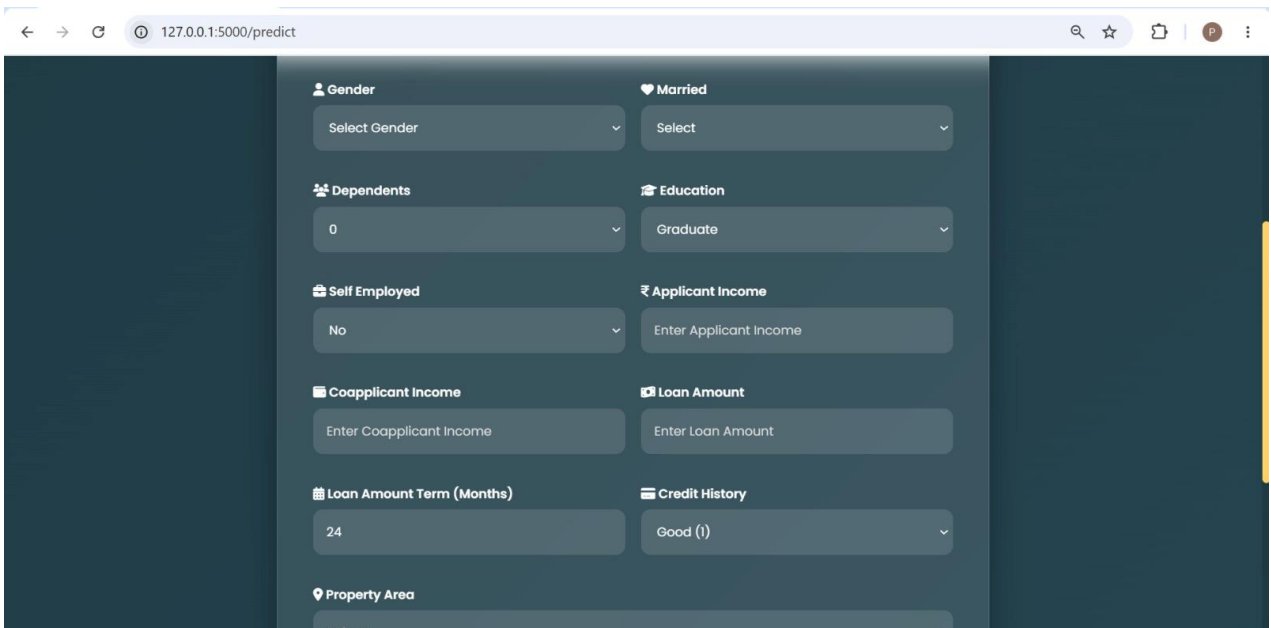
Property Area: Urban

[Predict Loan Approval](#)

[Back to Home](#)

Smart Lender AI © 2026

Prediction Page:



127.0.0.1:5000/predict

Gender: Select Gender

Married: Select

Dependents: 0

Education: Graduate

Self Employed: No

Applicant Income: ₹ Enter Applicant Income

Coapplicant Income: Enter Coapplicant Income

Loan Amount: Enter Loan Amount

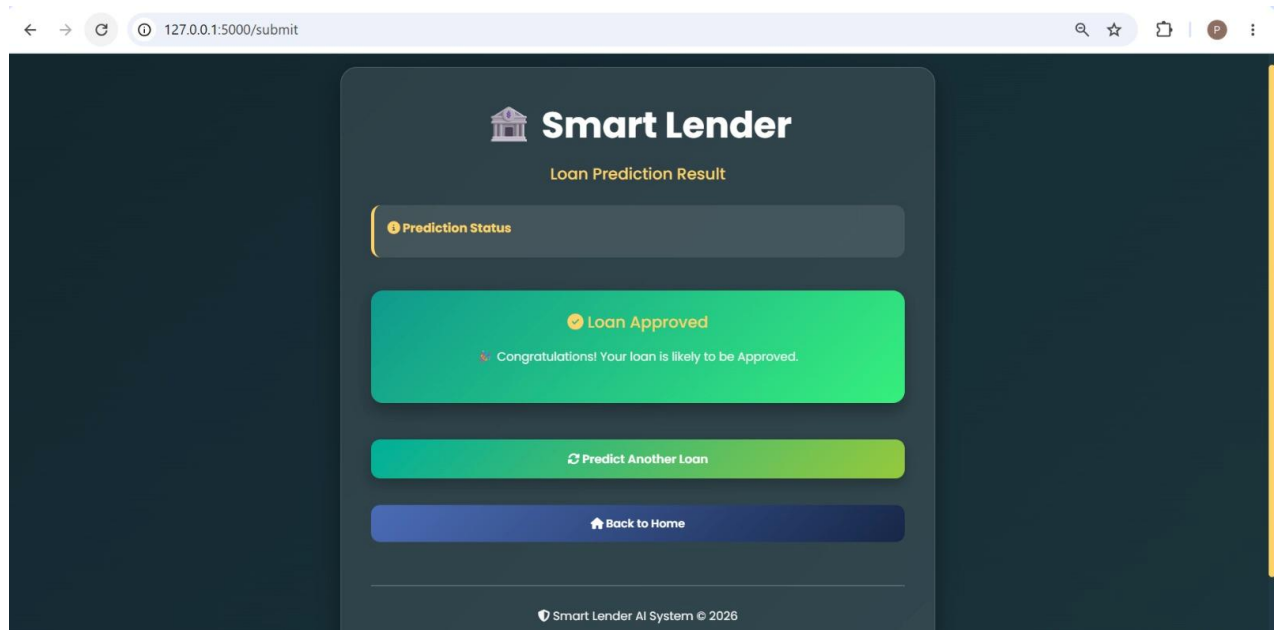
Loan Amount Term (Months): 24

Credit History: Good (I)

Property Area: Urban

Description: The Prediction Page contains the loan application form where users enter applicant details such as Gender, Marital Status, Dependents, Education, Self-Employment Status, Applicant Income, Co-applicant Income, Loan Amount, Loan Amount Term, Credit History, and Property Area. After filling in the required information, users click the **Submit** button to send the data to the Flask application for loan approval prediction.

Results Section:



Description: The Result Page displays the loan approval prediction generated by the trained machine learning model. Based on the applicant details provided, the application displays whether the loan is **Approved** or **Not Approved**. The page also provides an option to return to the prediction page and evaluate another loan application.

Conclusion:

The **Smart Lender – Loan Approval Prediction System** successfully demonstrates the application of machine learning techniques in automating the loan approval process. By utilizing algorithms such as Decision Tree, Random Forest, K-Nearest Neighbors (KNN), and Gradient Boosting, the system accurately predicts loan approval based on applicant information. The integration of the trained machine learning model with the Flask web framework provides a simple, efficient, and user-friendly platform for real-time loan prediction.

The project covered all major stages of the machine learning lifecycle, including data collection, exploratory data analysis, data preprocessing, feature engineering, model training, evaluation, and deployment. These processes helped improve prediction accuracy and ensured the development of a reliable loan prediction system.

In the future, the Smart Lender application can be enhanced by integrating real-time credit score verification, user authentication, database support, cloud deployment, and advanced machine learning models to further improve prediction accuracy, scalability, and overall system performance.